■■ Audixa — Project Implementation Plan & Technical Report

Stage: Post-30% Implementation & 60% Milestone Review | **Tech:** Python, Flask, Faster-Whisper, WaveSurfer.js, SQLite

---

1. Executive Summary & Milestone Progress

Audixa is a dedicated, web-based speech synchronization and content structuring platform designed to convert lengthy audio and video recordings into interactive transcripts, time-synchronized waveforms, and auto-generated video chapters. Following the initial 30% UI scaffolding milestone, the platform has reached a fully functional **60% working state** with an active local AI speech recognition pipeline, 3-panel synchronized editor, multi-format exports, and database persistence.

Feature / Subsystem	Initial 30% Baseline	Post-30% Implementation (Current)
Speech-to-Text	Static JSON mockup fixtures	**Live local Faster-Whisper AI** (int8 quantization, CTranslate2 engine).
Inference Speed	N/A (no model execution)	**3x–5x acceleration** via multi-threading, greedy 1-beam search, VAD filtering.
Audio Player	Basic static canvas container	**Bi-directional WaveSurfer sync**, speed pitching (0.5x–2x), volume slider, mute.
Transcript Editor	Read-only placeholder text	**Live active segment highlight**, smooth auto-scroll, in-line editing, click-to-seek.
Chapter Maker	Wireframe cards only	**Pause-gap (>=1.5s) auto-detection**, manual '+ Add Chapter', inline rename.
Export Engine	Disabled UI dropdown menu	**6 Live Formats**: TXT, SubRip (.srt), WebVTT (.vtt), YouTube Chapters, Markdown, JSON.
Database Storage	Basic schema prone to collisions	**Scoped IDs**, INSERT OR REPLACE transactions, foreign key cascades, manual saves.
UI/UX Design	Cluttered with debug badges	**Streamlined Organic Warm Sage layout** with eliminated button clutter.

2. AI Speech Recognition & Speed Architecture

To replace the static fixtures from Phase 1, Audixa integrates **Faster-Whisper**—a high-efficiency CTranslate2 reimplementation of OpenAI's Whisper model optimized for fast CPU execution.

- **Multi-Threaded CPU Scaling:** Automatically detects all available CPU cores using os.cpu_count() (utilizing 12 hardware threads) for parallelized tensor operations.
- **int8 Quantization:** Compresses neural weights from float32 to 8-bit integers, halving memory bandwidth and accelerating execution on standard CPU hardware.
- **Greedy 1-Beam Search:** Sets beam_size=1 and best_of=1, reducing decoding latency by ~70% over beam-5 search while preserving word accuracy.
- **Voice Activity Detection (VAD):** Integrates Silero VAD (vad_filter=True) to discard non-speech silence and prevent repetitive looping.

- **Background Model Pre-Warming:** Spawns a background thread on server startup to load weights into memory, ensuring zero cold-start delay for the first user upload.
- **Language Support:** Provides dedicated modes for English (en), Hindi (hi), and mixed bilingual Hinglish (auto).

3. Synchronized 3-Panel Editor Architecture

The core studio interface (editor.html & editor.js) delivers a multi-panel workspace where audio playback, transcription text, and chapter markers are kept in real-time synchronization.

Panel	Core Features & Technical Implementation
Panel A: **Audio Waveform & Player**	• Interactive canvas waveform generated via **WaveSurfer.js v7**. • Real-time timeupdate event dispatching. • Play/Pause with spacebar shortcut. • Speed selector: 0.5x, 0.75x, 1.0x, 1.25x, 1.5x, 2.0x. • Real-time volume slider and one-click mute toggle.
Panel B: **Interactive Transcript**	• **Live Active Segment Highlighting:** Compares audio playhead to segment intervals and applies high-contrast active styling. • **Auto-Scroll:** Smoothly scrolls dialogue into view as speaker advances. • **In-Line Editing:** Editable segment text via contenteditable="true". • **Click-to-Seek:** Clicking timestamp badges seeks audio instantly. • **Instant Filter:** Live client-side phrase search with match counter. • **Segment Deletion:** Single-click segment deletion.
Panel C: **Auto-Chapter Maker**	• **Pause-Gap Detection:** Heuristic scans speech pauses (≥ 1.5s) to propose logical topic transitions. • **Manual Creation:** + Add Chapter creates a chapter at the exact playhead position. • **In-Line Title Renaming:** Direct text input for custom chapter names. • **Click-to-Seek:** Clicking a chapter jumps audio directly to that topic.

4. UI/UX Streamlining & Decluttering (Post-30% Refinements)

To provide a clean, distraction-free editing environment, several redundant buttons, auxiliary badges, and repetitive indicators were removed per interface evaluation:

- **Removed Skip Buttons:** Removed -5s and +5s buttons from the audio player controls.
- **Removed Auto-saved Text:** Cleaned duplicate status text underneath the chapters panel.
- **Cleaned Chapter Actions:** Replaced the auxiliary Auto-Detect button and dynamic + Add at 01:27 label with a single clean **+ Add Chapter** button.
- **Removed Header Badges:** Removed Auto / Hinglish, ■■ duration, ✓ Saved, and ■ History pills from the top navigation bar.
- **Cleaned Segment Cards:** Removed #index labels and ⋗■ Split / ■ Merge buttons from segment card headers.
- **Cleaned Transcript Toolbar:** Removed 123 words counter, + Add Segment, and find-replace popups, retaining direct in-line editing and search.

5. Multi-Format Export Engine

Audixa provides direct file downloads via /api/export/<id>/<format> supporting 6 standard media workflows:

Format	Extension	Output Specification	Primary Use Case
Plain Text	.txt	Clean transcript text with timestamps.	Documentation, study notes.

SubRip Subtitles	.srt	Numbered subtitle blocks with comma ms timestamps.	Premiere, DaVinci, Final Cut.
WebVTT	.vtt	HTML5 subtitle track format with dot ms timestamps.	Web video players & LMS.
YouTube Chapters	.txt	Standard timestamp list (00:00 Intro\n01:15 Topic 1).	YouTube video descriptions.
Markdown Summary	.md	Formatted doc with headings, metadata, and blockquotes.	Notion, GitHub, Obsidian.
Full JSON	.json	Complete structured schema with segment arrays.	API integrations & pipelines.

6. Database & REST API Architecture

Audixa utilizes SQLite (audixa.db) with 3 relational tables (transcripts, segments, chapters) featuring foreign key cascade deletions and scoped ID keys (transcript_id_raw_id) using INSERT OR REPLACE to prevent primary key collision errors.

Method	Route	Description & Output
POST	/upload	Uploads media, runs Whisper AI & pause chaptering, returns transcript ID.
GET	/transcript/<id>	Returns metadata, segments list, chapters list, and audio URL.
POST	/save_transcript/<id>	Persists updated segment text and timestamps into SQLite.
POST	/save_chapters/<id>	Persists updated chapter titles and timestamps into SQLite.
GET	/api/export/<id>/<fmt>	Streams exported file (txt, srt, vtt, youtube, md, json).
GET	/audio/<id>	Streams raw audio with HTTP 206 Partial Content byte ranges.
GET	/api/health	Health check returning backend and engine readiness.

7. Automated Testing & Verification

All 8 automated backend and integration test suites pass with 100% success rate (Ran 8 tests in ~3.5s: OK):

- **test_01_db_crud:** Verified SQLite insertion, queries, and foreign key cascades.
- **test_02_routes_get:** Verified HTTP 200 responses on / and /editor.
- **test_03_api_endpoints:** Verified JSON API health and metadata endpoints.
- **test_04_upload_flow:** End-to-end Whisper speech recognition test (10.3s audio in 2.8s).
- **test_05_segment_split_and_merge:** Verified split/merge data consistency.
- **test_06_regenerate_chapters:** Verified pause-gap chaptering heuristic.
- **test_07_export_endpoints:** Verified byte streaming for all 6 export formats.
- **test_08_history_and_health:** Verified history fetching and deletion cascades.

8. Viva & Faculty Review Q&A; Guide

Q1: What major progress has been made since the 30% milestone?
A: We transitioned from static mockups to a live local AI speech recognition pipeline (Faster-Whisper), built the synchronized 3-panel editor with active highlight and auto-scroll, implemented the pause-gap chaptering heuristic, built 6 export formats, and hardened the SQLite database.

Q2: How is transcription made fast on local CPUs?
A: Using CTranslate2 with int8 quantization, multi-threading across all 12 CPU hardware threads, greedy 1-beam search, Silero VAD silence filtering, and background model pre-warming.

Q3: How does the editor synchronize playback with transcript?
A: WaveSurfer dispatches timeupdate events; the editor compares the playhead against segment boundaries [start_time, end_time], applies the active CSS class, and auto-scrolls smoothly.